Fundamentals of Programming Languages

Exercises 5

Simply Typed Lambda Calculus

Mestrado em Engenharia Informática
Faculdade de Ciências da Universidade de Lisboa

2026/2027

The exercise numbers in this list refer to the exercise numbers in book Types and Programming Languages. Benjamin Pierce, The MIT Press, 2002.

Exercises in TAPL

1. Draw derivation trees for given terms (Exercise 9.2.2).
2. Find a context (Exercise 9.2.3).
3. Find a type for xx (Exercise 9.3.2).
4. Is there a closed term of type A? Of type $A \to A$? Of type $(A \to A) \to A$? Justify.
5. Theorem 9.3.3 (Uniqueness of types) says that a term has at most one type in a given context. What would go wrong if abstractions carried no type annotation, that is, if we wrote $\lambda x.t$ instead of $\lambda x{:}T.t$? How would rule T-Abs change? Give a term that would then have infinitely many types.

Exercises in SASyLF

6. Prove that no closed value has type A: if t is a value and $\vdash t : A$, then we have a contradiction.
7. Write SASyLF typing derivations for the Church encodings `tru`, `fls` and `fst` of the Implementation exercises below. Notes: write variables as `x1`, `x2`, ...
8. Write SASyLF proofs for items 2 and 3 above.
9. Define multi-step evaluation $t \to^* t'$ as a judgment with two rules (Definition 3.5.9 and Exercise 3.5.10). Prove that it preserves types: if $\Gamma \vdash t : T$ and $t \to^* t'$, then $\Gamma \vdash t' : T$. Compare with Exercise 2.2.8.

10. The SASyLF proof of Theorem 9.3.9 (Preservation) is by induction on the evaluation derivation. Write an alternative proof, by induction on the typing derivation, as in TAPL.

11. Prove Theorem 9.3.3 (Uniqueness of types): if $\Gamma \vdash t : T$ and $\Gamma \vdash t : T'$, then $T = T'$. Suggestion: define a judgment for type equality, `T = T'`, with a single rule, reflexivity.

12. Prove that evaluation is deterministic (compare with Theorem 3.5.4): if $t \to t'$ and $t \to t''$, then $t' = t''$. Suggestion: define a judgment for term equality, as for types, and prove first that values do not evaluate.

Implementation The following exercises build on the Haskell source code adapted from TAPL and provided at the course website.

13. Study the data types `Term` and `Type` in module `TypedLambda`.

14. Complete function `typeof`, which computes the type of a term in a given context. Refer to Figure 9-1. Use it to check the types of the terms you write in the exercises below.

15. What does `typeof` do on the term `Abs "x" A (App (Var "x") (Var "x"))`? Explain, relating to Exercise 9.3.2.

16. Try writing $\Omega = (\lambda x.x\ x)\ (\lambda x.x\ x)$ as a typed `Term`. Compare with module `UntypedLambda`, where `eval` loops on Ω. Can `eval` loop on a term accepted by `typeof`? (See Chapter 12.)

17. Define `boolT :: Type`, the type of Church booleans.

18. Write `Terms` for `tru`, `fls` and `test`. Take inspiration from Chapter 5, but provide adequate types.

19. Write `Terms` for `and`, `or` and `not`. Note that λ`m`.λ`n`.`m n fls` is not typable: `m` would choose between two booleans, not two terms of type `A`. You may consider using instead λ`m`.λ`n`.λ`t`.λ`f`.`m` (`n t f`) `f`.

20. Now consider `App (App and tru) fls`. Does it evaluate (using the `eval` function) to `fls`? If not, why? How do the two normal forms differ?

21. Define `pairT :: Type`, the type of pairs of values of type `A`.

22. Write `Terms` for `pair`, `fst` and `snd`. Again, take inspiration from Chapter 5, but provide adequate types.

23. Can `pair` build a pair whose first component has type `A` and whose second component has type `boolT`? Why not? (Section 11.6 introduces product types.)

24. Write `Terms` for the first few Church numerals, `c0`, `c1` and `c2`. Again, provide adequate types.

25. Chapter 5 notes that `c0` and `fls` are the same term. Are they still the same with types? Use `typeof`.

26. Write `Terms` for `scc` and `plus`. Then try `times`, defined in Chapter 5 as λ`m`.λ`n`.`m` (`plus` `n`) c_0. Why is it not typable?

27. Check Theorem 9.3.9 (Preservation) on examples: for terms such as `App (App and tru) fls` and `App (App plus c1) c1`, verify that `typeof` gives the same type to every term along the evaluation sequence produced by `step`.

28. Write a function `erase :: Term → U.Term` that removes the type annotations from a term (Definition 9.5.1), where `U` is your module `UntypedLambda` from Exercises 4, imported qualified. Check on your examples that `erase (eval t)` and `U.eval (erase t)` coincide (Theorem 9.5.2), up to the renaming of bound variables.

Vasco T. Vasconcelos
October 16, 2026